SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)

ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Name : P.VENKATA SANDEEP

Reg. No : 11249A290

Class : II B.E. (CSE)

Course Code: BCSF244P90

Course Name: OPERATING SYSTEM LAB

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)

ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by **Mr/Ms.** **POTHALA VENKATA SANDEEP**

with **Reg. No** **11249A290** of **II-B.E.(CSE)** during the academic year 2025 – 2026.

Station:Enathur

Date: 21-04-2026

Staff-in-charge **Head of the Department**

Submitted for the Practical examination held on _____ .

Examiner-1 **Examiner-2**

INDEX

SL. NO	NAME OF THE EXERCISE	DATE	PAGE NO	SIGNATURE
1	Installation of windows Operating System	13-02-26	-	
2	Practice the following Linux commands a. File and Directory Related Commands b. Process and Status Information Commands c. Text Related Commands d. File Permission Commands	13-02-26	1-3	
3	Execute a Shell Program a. To find the whether a Number is even or odd b. To find the biggest of two numbers c. To find the biggest of three numbers d. To find the factorial of a Number e. To display Fibonacci Series	20-02-26	4-6	
4	Practice the Linux Pipes and Filters commands a. Grep, b. Sed c. Awk commands	06-03-26	7	
5	Implement System Calls using C a. Stat() b. Wait() c. Getpid() d. Opendir(), readdir() e. Open(), Read(), Write()	13-03-26	8-10	
6	Implement Process Management a. Fork() b. Exec()	13-03-26	11-12	

7	Implement various CPU Scheduling algorithm using C a. FIFO b. Round Robin c. SJF	20-03-26	13-17	
8	Write C programs to avoid Deadlock using Banker's Algorithm	03-04-26	18-20	
9	Write C programs to implement the following Memory Allocation Methods a. First Fit b. Worst Fit c. Best Fit	03-04-26	21-25	
10	Install any guest operating system like Linux using VMware	03-04-26	-	

Exp: 02
Date: 03-02-26 Practice linux commands.

a.) file and Directory Related commands.
these commands help you manage files & directories

P. venkata sandeep 1249120 Page No.

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1	3/2/26	Installation of windows operating system		
2.)	03-02-26	Practicing linux commands		
3.	20-02-26	Execute a shell program even(or) odd, Biggest of two numbers, three numbers, factorial of a numbers, Fibonacci series.		
4.	06-03-26	Practice the Linux pipes & filters commands. a.) Grep. b. sed, c. Awk commands		
5.).	13-03-26	implement system calls using C. a.) stat() b.) wait() c.) get pid() d.) open dir() readdir() e.) open(), Read(), write.		
6.).	13-03-26	Implement Process managment a. fork() b. exec()		
7.).	20-03-26	Implement various cpu scheduling algorithm using C. a.) FIFO b.) Round Robin c.) SSF.		
8.)	03-04-26	write c Program.		

		to avoid. Dead lock using Bankers Algo rithm.
a.)	03-04-26	write c programs to implement the following memory Allocation methods a.) first fit b.) worst fit c.) Best fit.
b.)	~~02~~-04-26	Install any guest operating system like Linux using vmware.

Completed
Rohith
7/4/26

Date: 03-02-26 Practice Linux commands.

a.) file and Directory Related commands.

These commands help you manage files & directories in the Linux file system

→ ls - list directory contents.

ex → ls → lists files and directories in the current directory.

- cd - change directory

ex - cd /home/user/Documents → navigates to the Document folder.

- Pwd - Print working directory

Ex - Pwd → shows the full Path of your current directory

- mkdir - make a directory

ex - mkdir new_folder → creates a new folder named new_folder.

→ rm - Remove files or directories.

Ex - rm file.txt → deletes file.txt

- cp - copy files or directories.

- mv - move or rename files or directories

ex - mv file.txt new file.txt.

- touch - create an empty file or update the timestamp of a existing file

b.) Process & status information commands

→ Ps - Display current Process

→ ex - PS → shows Process running under your current user.

→ top - Task manager (real-time view of system Process)

> kill - kill a Process by PID
ex : kill 1234 → terminates Process with PID
> df - Disk space usage.
ex : df -h → shows disk space
> free - memory usage
> ex - free -h → show memory usage
> uptime - shows how long the system been running.
> who - shows who is logged into the system
> Text Related commands,
> cat - concatenate, display files
ex → cat file.txt
> more - view the content of files page-by-page
> less - view the content of files with scroll options.
> grep - search for text within files
ex : grep "error" log.txt
> wc - word count
ex : wc file.txt
> head - view the first few lines of a file
> tail - view the last few lines of a file
> tail -n 10 file.txt
> sed - stream editor for text manipulation
> sed 's/old/new/g' file.txt
> awk - Pattern scanning and Processing Language

d. file permision commands
> chmod - change file Permissions.
ex : chmod 755 file.txt
> chown - change file ownership
> chgrp : change group ownership
> ex : chgrp group file.txt
> umask - set default file creation Permissions
> ex : umask 022
> stat : Display file (or) filesystem status
ex : stat file.txt

output:
Enter a number = 10
10 is even

output: Enter first number: 9
Enter second number: 6
9 is bigger

Ex. No. 3
Date: 20-02-26 Executing a shell Program

a) To find whether a Number is Even (or) odd

```
Read a no
echo "Enter a number"
read num
if [ $((($num %2))-eq 0]; then
   echo "$num is Even"
else
   echo "$num is odd"
fi
```

b) To find the biggest of Two numbers

```
echo "Enter first number"
read num1
echo "Enter second number"
read num2
if [$num1 -gt $num2]; then
   echo "$num1 is the biggest"
elif [$num1 -lt $num2]; then
   echo "$num2 is the biggest"
else
   echo "Both numbers are equal"
fi
```

c. To find the Biggest of three numbers

```
echo "Enter first number"
read num1
echo "Enter second number"
```

output: enter first number : 2
enter second number : 8
enter third number : 4
8 is the biggest

output: enter the number of terms : 7
factorial of 7 is 2

```
read num2
echo "enter third number"
read num3
if [ $num1 -gt $num2 ] && [ $num1 -gt $num3 ] then
echo "$num1 is the biggest"
elif [ $num2 -gt $num1 ] && [ $num2 -gt
$num3 ]; then
echo "$num2 is the biggest"
else
echo "$num3 is the biggest"
f) To find the factorial of a number
echo "Enter a number"
read num
factorial=1
for ((i=1; i<=$num; i++))
do factorial=$((factorial*i))
done
echo "Factorial of $num is $factorial"
e) To Display fibonacci series
echo "Enter the number of terms"
read terms
a=0,
```

output: ~~the Fibonacci series~~

~~to~~ enter no. of terms : 7

fibonacci series : 0 1 1 2 3 5 8

```
echo "Fibonacci series."
for (((i=0; i<$terms ; i++))
do echo -n "$a"
next=$((a+b))
a=$b
b=$next
done
echo.
```

Result: the executing of shell program has done & output is verified.

output: enter the log file to search: log
enter the search term: echo
echo: enter the log file to search
echo: enter the search term

output: enter file to search
first.txt
enter old_word
end
enter new word
an endeep has been replaced with
the word end, in first.txt.
endeep: 1

output: enter the filename:
first.txt

aim: Practice the linux Pipes & filter commands.

a) sed:
```
echo "Enter the log file to search"
read filename
echo "enter search term"
read search_term
grep "$search_term" $filename
```

b) sed
```
echo "Enter the filename"
read filename
echo "Enter the word to replace"
read old_word
echo "enter the new word
read new_word
sed -i "s/$old_word/$new_word/g" $filename
echo "the word '$old_word' has been replaced
with '$new_word'" in $filename
```

c) awk:
```
echo "Enter the filename"
read filename
awk '{print $1 $3}' $filename
```

Result: Practicing the linux Pipes & filter's commands & output is verified.

output: usage: % output <text>
% output doc.txt

file size : 230 bytes
number of links = 1
file inode : 1456799
file Permisions : 664

Ex: 05
Date: 13-03-25

Implement system calls using C

a) stat() system call

```
#include <stdio.h>
#include <sys/stat.h>
int main(int argc, char *argv[])
{ struct stat file stat;
  if (argc < 2) {
      Printf ("usage: %s < filename>\n",
        argv[0])
      return 1;
  }
  if (stat(argv[1], &file stat) == -1){
  Perror ("stat")
  return 1;
}
Printf ("File size: %ld bytes\n", file
  stat.st_size);
Printf ("Number of Links: %ld\n", file
  stat.st_nlink);
Printf ("file inode: %ld\n", filestat.
  st_ino);
Printf ("file Permisions: %O\n", filestat.
st_mode & 0777);
return 0;
}
```

output:- child Process executing
child Process exiting
Parent resumed after child term

output:- Process id: 3177
Parent id : 3134

2) wait() system call

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
 pid = *pid;
 Pid = fork();
 if (pid == 0) {
  Printf("child Process executing \n");
  sleep(5)
 Printf("child Process existing \n");
 }
 else
 {
  wait(NULL);
 Printf("Parent Process resumed after
   child termination \n");
 }
 return 0;
}
```

3) get pid() system call

```
#include <stdio.h>
#include <unistd.h>
int main()
{
 Printf("Process ID: %d \n", getpid
 Printf("Parent Process ID: %d \n
      getppid());
 return 0;
}
```

output :- m/c dir 2 ~~ls~~
contents of directory
one.txt, first.txt, evenmish ....

6) opendir() and readdir() system calls.

```
#include <stdio.h>
#include <dirent.h>
int main (int argc, char* argv[])
{DIR *dir;
struct dirent* entry;
if (argc<2){
  Print ("usage: %s <directory path
  > /n", argv[0]);
  return 1;
}.
dir = opendir (argv[1]);
if (dir= NULL){
   Perror ("opendir").
   return 1;
}.
Printf ("contents of directory : \n");
while ((entry= readdir (dir))!=NULL)
Printf ("%s\n" entry->d_name);
}
close dir (dir);
return 0;
}.
```

Result :- Implementation system calls using has been done & output is verified.

output: Parent Process:

Parent PID = 3555
child PID = 3556

child Process:

child PID: 3556
Parent PID: 3555

Exc: 06
Date: 13-03-26 Implement Process Managment.

fork() system call

```
#include <stdio.h>
#include <unistd.h>
int main()
{pid_t Pid;
Pid = fork();
if (pid < 0){
        printf("Fork failed \n");
}
else if (pid == 0){
Printf ("child Process \n");
Printf ("child PID: %d \n", getpid());
Printf ("Parent PID: %d \n", getppid());
}
else {
 Printf ("Parent Process \n");
Printf ("Parent PID: %d \n", getpid());
Printf ("child PID: %d \n", Pid);
}
return 0;
}
```

b) execl() system call

```
#include <stdio.h>
#include <unistd.h>
int main().
```

output enter number of processes : 2
enter burst times : 10
enter time quantum : 6

Process	BT	WT	TAT
P_1	6	0	6
P_2	5	1	6

Average waiting time : 0.50
Average Turnaround time : 3.50

```
Printf("%d\t %d\t %d\t %d\t\n", i+1, bt[i]
    wt[i], tat[i]);
}
Printf("Average waiting time = %f\t\n", avg_wt);
Printf("Average Turnaround time = %f\t\n",
                                avg_tat);
return 0;
}
```

b) Round Robin Scheduling

```
#include <stdio.h>
int main()
{int n, i, tq;
int bt[10], rt[10], wt[10], tat[10];
int time=0, done;
float avg_wt=0, avg_tat=0;
Printf ("Enter number of Process:");
scanf ("%d", &n);
Printf ("Enter burst time:\n");
  for (i=0; i<n; i++){
    scanf ("%d", &bt[i]);
    rt[i] = bt[i];
    wt[i] =0;
}
Printf ("Enter time quantum:");
```

output (FCFS) :-

Enter number of Process : 2

Enter burst times : 1 3

Process	B.T	WT	TAT
		0	1
P1	1	1	4
P2	3		

Average waiting time :- 0.50

Average Turnaround time : 2.50

Exp 07 — Implementation various CPU schedulingalgorithm using C.

Date :- 20-09-24

FIFO / FCFS (first come first serve)

```
#include <stdio.h>
int main()
{int n,i;
int bt[10],wt[10],tat[10];
float avg_wt=0, avg_tat=0;
Printf ("Enter number of Processes:");
scanf ("%d", &n);
Printf ("Enter burst times: \n");
for (i=0; i<n; i++){
  scanf ("%d", &bt[i]);
}
wt[0]=0;
for(i=1; i<n; i++){
  wt[i] = wt[i-1] + bt[i-1];
}
for (i=0; i<n; i++){
  tat[i]= wt[i]+ bt[i];
  avg_wt += wt[i];
  avg_tat += tat[i];
}
Printf ("\nProcess \tBT \tWT \tTAT\n");
  for (i=0; i<n; i++){
```

output: Parent Process
child Process before exec
ls -l
5.C. 2d.sh 3e.sh
4b.sh

```
{pid_t rid;
pid = fork();
if (pid == 0){
  Printf("Child Process before exec()\n")
  execl("/bin/ls", "ls", NULL);
Printf("this line will not be exec
  \n");
}
else{
  Printf("parent Process \n");
}
return 0;
}
```

Result: Implementation of Process managment has done & output is verified

```
scanf("%d", &tq);
do {
  done = 1;
  for (i=0; i<n; i++){
  if (rt[i] > 0) {
    done = 0;
  if (rt[i] > tq) {
  time += tq;
  rt[i] -= tq;
  }
  else {
    time += rt[i]
    wt[i] = time - bt[i]
      rt[i] = 0;
    }
  }
}while (!done);
for (i=0; i<n; i++){
tat[i] = bt[i] + wt[i];
avg-wt += wt[i];
avg-tat += tat[i];
}
Printf("\n Process | BT | WT | TAT \n");
for (i=0; i<n; i++){
  Printf(" %d | %d | %d | %d \n
```

Exp:- 08
Date:- 03-04-26

To avoid Deadlock using Banker Algorithm.

```
#include<stdio.h>
int main()
{int n,m,i,j,k;
 int alloc [10] [10], max [10] [10],
need [10]. [10];
int avail [10], finish [10], safe seq [10]
Print f ("Enter number of Process:\n);
scan f ("%d", &n);
Print f ("Enter number of resources:\n);
Scan f ("%d", &m);
Print f ("Enter Allocation matrix:\n);
for (i=0 ; i<n; i++);
   for (i=0 ; i<m; i++);
      scanf ("%d", &alloc [i] [j]);
Print f ("Enter maximum matrix:\n);
for (i=0; i<n; i++)
 for (j=0; i<n; j++)
   scan f ("%d", &max [i] [j])
Print f "Enter Available Resources: \n);
    for (i=0; i<n; i++
    scan f ("%d", &avail [i])
    for (i=0 ; i<n; i++).
```

```
        bt[i] = bt[j];
    }
    bt[i] = temp;
  }
}
wt[0] = 0;
for (i=1; i<n; i++) {
    wt[i] = wt[i-1] + bt[i-1];
}
for (i=0; i<n; i++) {
tat[i] = bt[i] + wt[i];
avg_wt += wt[i];
avg_tat += tat[i];
}
Printf("\n Process \t BT \t WT \t TAT \n");
for (i=0; i<n; i++) {
Printf("p%d \t %d \t %d \t %d \n", i+1,
        bt[i], wt[i], tat[i]);
}
Printf("Average waiting Time= %f\n",
        avg_wt/n);
printf("Average Turnaround time= %f\n",
        avg_tat/n);
return 0;
}
```

Result:- Implementing of various cpu scheduling algorithm using c has been done.

that is order number of Process
order burst times : 4 3

Process	B.T	W.T	T.A.T
P_1	1	0	1
P_2	3	1	4

Average waiting time : 0.50
Average Turnaround time : 2.5

```
printf("Average waiting time: %f\n",
        avg_wt/n);
printf("Average Turnaround time:
        %f\n", avg_tat/n);
return 0;
}
```

a) SJF (Shortest Job First)

```
#include<stdio.h>
int main()
{int n, i, j;
int bt[10], wt[10], tat[10], temp;
float avg_wt = 0, avg_tat = 0;
Printf("Enter number of Process:")
scanf("%d", &n);
Print f("Enter burst times:\n");
for (i=0; i<n; i++){
  scanf("%d", &bt[i]);
}
for (i=0; i<n-1; i++){
    for (j=i+1; j<n; j++){
      if (bt[i] > bt[j]){
        temp = bt[i];
```

```
Printf ("\n system is in SAFE state\n
        safe sequence :");
  for (i=0 ; i<n ; i++)
    Printf ("%d "; safe seq [i]);
  Printf ("\n");
  return 0;
}
```

Result : Banker's Algorithm using c is done & output verified.

output

enter number of process : 3

enter no. of resources : 3

enter Allocation matrix

```
1 1 0
2 2 4
4 3 5
2 9 0
6 8 0
7 0 0
```

enter maximum matrix

```
1 1 0
2 2 4
3 3 5
4 6 6
7 8 9
3 9 4
```

enter available Resource =?

```
2 1 4
3 4 3
```

system is in safe state

safe sequence : P0 P1 P2 P4 P3

```
for (i=0; i<m; i++)
  need [i][j] = max [i][j] - alloc [i][j]
for (i=0; i<m; i++)
  finish [i] = 0
int count = 0;
while (count<m) {
  int found = 0;
for (i=0; i<m; i++) {
  if (finish [i] == 0) {
for (j=0; j<n; j++)
  if (need [i][j] > avail [j])
    break;
if (j==n) {
  for (k=0; k<n; k++)
    avail [k] = avail [k] + alloc [i] [k]
    safe seq [count++] = i;
    finish [i] = 1;
    found = 1;
  }
}
}
if (!found) {
  Printf ("\n system is in unsafe
          (Dead Lock Possible)\n");
  return 0;
}
}
```

enter number of blocks : 5

enter size of each block

100, 200, 300, 500, 600

enter no. of Process : 4

enter size of each Process

212 417 112 426

Process	size	Block
P1	212	3
P2	417	3
P3	112	3
P4	426	4

Ex: 19

Date: 3-4-20 Implementing the memory Allocation methods.

a) first fit Program.

```
#include <stdio.h>
int main()
{ int np, nb, i;
  int b[10], p[10], alloc[10];
Print f ("Enter number of blocks:");
scan f ("%d", &nb);
Print f ("Enter size of each block:\n");
  for (i=0; i<nb; i++)
  scan f ("%d", &b[i]);
Print f ("Enter number of Process:");
  scan f ("%d", &np);
Print f ("Enter size of each Process:\n");
  for (i=0; i<np; i++)
    scan f ("%d", &p[i]);
  for (i=0; i<np; i++)
    alloc [i]=-1;
  for (i=0; i<np; i++){
  for (j=0; j<nb; j++){
    if (b[j] >= p[i]){
      alloc[i] = j;
      b[j] -= p[i];
      break;
```

output: enter number of blocks : 5

enter size of each block

100 500 200 300 600

Enter no. of Process : -4

enter size of each Processor :-

312 417 112 426

Process	size	Block
P1	312	5
P2	417	5
P3	112	4
P4	426	3

```
Printf ("\n Process \t size \t Block \n");
  for (i=0; i<np; i++) {
    if (alloc[i] != -1)
Printf ("P %d \t %d \t %d \n", i+1, p[i],
                alloc[i]+1);
else:
  Printf ("P %d \t %d \t Not Allocated \n",
  i+1, p[i]);
  }
  return 0;
  }
```

b) worst fit Program:

```
#include <stdio.h>
int main()
{ int nb, np, i, j;
int b[10], p[10], alloc[10];
int max; pos;
Printf ("Enter number of blocks:");
scanf ("%d", &nb);
Printf ("Enter size of each block:
        \n");
for (i=0; i<nb; i++);
scanf ("%d", &b[i]);
Printf ("Enter number of Processes:")
scanf ("%d", &np).
```

output: enter number of blocks : 5
Enter size of each block:
100 500 200 300 600
Enter no. of Processes : 4
Enter size of each Process
212 417 112 426

Process	size	Block
P1	212	2
P2	417	5
P3	112	2
P4	426	not allocated

2) Best fit Algorithm:

```
#include<stdio.h>
int main()
{
  int nb, np, i;
  int b[20], p[20], alloc[20];
  int min, Pos;
  Printf ("Enter number of blocks: ");
  scanf ("%d", &nb);
  Printf ("Enter size of each block: \n");
  for (i=0; i<nb; i++)
      scanf ("%d", &b[i]);
  Printf ("Enter number of Processes: ");
  scanf ("%d", &np);
  Printf ("Enter size of each Process: \n");
  for (i=0; i<np; i++)
      scanf ("%d", &p[i]);
  for (i=0; i<np; i++)
      alloc [i] = -1;
  for (i=0; i<np; i++) {
      min = 9999;
      Pos = -1;
      for (j=0; j<nb; j++){
          if (b[j] >= p[i] && b[j] < min)
          {
              min = b[j]
              Pos = j;
          }
          if (pos != -1)
          {
              alloc [i] = pos;
```

```
Printf ("Enter size of each Process:\n");
for (i=0; i<n; i++)
    scanf ("%d", &p[i]);
for (i=0; i<np; i++)
    alloc [i] = -1;
for (i=0; i<np; i++) {
  max = -1
  pos = -1
  for (j=0; j<nb; j++){
    if (b[j] >= p[i] && b[j] > max){
      max = b [j]
      pos = j;
    }
  }
  if (pos != -1) {
      alloc [i] = pos;
      b [pos] -= p[i];
  }
}
Printf ("\n Process \t size \t Block \n");
for (i=0; i<np; i++) {
    if (alloc [i] != -1)
        Printf ("P%d \t %d \t %d \n", i+1, p[i],
    alloc [i] +1);
    else
      Printf ("P%d \t %d \t Not Allocation \n",
    i+1, p[i]);
}
return 0;
}
```

> kill - kill a Process by PID
ex : kill 1234 → terminates Process with PID
> df - Disk space usage.
ex : df -h → shows disk space
> free - memory usage
> ex - free -h → show memory usage
> uptime - shows how long the system been running.
> who - shows who is logged into the system
> Text Related commands,
> cat - concatenate, display files
ex → cat file.txt
> more - view the content of files page-by-page
> less - view the content of files with scroll options.
> grep - search for text within files
ex : grep "error" log.txt
> wc - word count
ex : wc file.txt
> head - view the first few lines of a file
> tail - view the last few lines of a file
> tail -n 10 file.txt
> sed - stream editor for text manipulation
> sed 's/old/new/g' file.txt
> awk - Pattern scanning and Processing Language

d. file permision commands
> chmod - change file Permissions.
ex : chmod 755 file.txt
> chown - change file ownership
> chgrp : change group ownership
> ex : chgrp group file.txt
> umask - set default file creation Permissions
> ex : umask 022
> stat : Display file (or) filesystem status
ex : stat file.txt

output:
Enter a number = 10
10 is even

output: Enter first number: 9
Enter second number: 6
9 is bigger

Ex. No. 3
Date: 20-02-26 Executing a shell Program

a) To find whether a Number is Even (or) odd

```
Read a no
echo "Enter a number"
read num
if [ $((($num %2))-eq 0]; then
   echo "$num is Even"
else
   echo "$num is odd"
fi
```

b) To find the biggest of Two numbers

```
echo "Enter first number"
read num1
echo "Enter second number"
read num2
if [$num1 -gt $num2]; then
   echo "$num1 is the biggest"
elif [$num1 -lt $num2]; then
   echo "$num2 is the biggest"
else
   echo "Both numbers are equal"
fi
```

c. To find the Biggest of three numbers

```
echo "Enter first number"
read num1
echo "Enter second number"
```

output: enter first number : 2
enter second number : 8
enter third number : 4
8 is the biggest

output: enter the number of terms : 7
factorial of 7 is 2

```
read num2
echo "enter third number"
read num3
if [ $num1 -gt $num2 ] && [ $num1 -gt $num3 ] then
echo "$num1 is the biggest"
elif [ $num2 -gt $num1 ] && [ $num2 -gt
$num3 ]; then
echo "$num2 is the biggest"
else
echo "$num3 is the biggest"
f) To find the factorial of a number
echo "Enter a number"
read num
factorial=1
for ((i=1; i<=$num; i++))
do factorial=$((factorial*i))
done
echo "Factorial of $num is $factorial"
e) To Display fibonacci series
echo "Enter the number of terms"
read terms
a=0,
```

output: ~~the Fibonacci series~~

~~to~~ enter no. of terms : 7

fibonacci series : 0 1 1 2 3 5 8

```
echo "Fibonacci series."
for (((i=0; i<$terms ; i++))
do echo -n "$a"
next=$((a+b))
a=$b
b=$next
done
echo.
```

Result: the executing of shell program has done & output is verified.

output: enter the log file to search: log
enter the search term: echo
echo: enter the log file to search
echo: enter the search term

output: enter file to search
first.txt
enter old_word
end
enter new word
an endeep has been replaced with
the word end, in first.txt.
endeep: 1

output: enter the filename:
first.txt

aim: Practice the linux Pipes & filter commands.

a) sed:
```
echo "Enter the log file to search"
read filename
echo "enter search term"
read search_term
grep "$search_term" $filename
```

b) sed
```
echo "Enter the filename"
read filename
echo "Enter the word to replace"
read old_word
echo "enter the new word
read new_word
sed -i "s/$old_word/$new_word/g" $filename
echo "the word '$old_word' has been replaced
with '$new_word'" in $filename
```

c) awk:
```
echo "Enter the filename"
read filename
awk '{print $1 $3}' $filename
```

Result: Practicing the linux Pipes & filter's commands & output is verified.

output: usage: % output <text>
% output doc.txt

file size : 230 bytes
number of links = 1
file inode : 1456799
file Permisions : 664

Ex: 05
Date: 13-03-25

Implement system calls using C

a) stat() system call

```
#include <stdio.h>
#include <sys/stat.h>
int main(int argc, char *argv[])
{ struct stat file stat;
  if (argc < 2) {
      Printf ("usage: %s < filename>\n",
        argv[0])
      return 1;
  }
  if (stat(argv[1], &file stat) == -1){
  Perror ("stat")
  return 1;
}
Printf ("File size: %ld bytes\n", file
  stat.st_size);
Printf ("Number of Links: %ld\n", file
  stat.st_nlink);
Printf ("file inode: %ld\n", filestat.
  st_ino);
Printf ("file Permisions: %O\n", filestat.
st_mode & 0777);
return 0;
}
```

output:- child Process executing
child Process exiting
Parent resumed after child term

output:- Process id: 3177
Parent id : 3134

2) wait() system call

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main()
{
 pid = *pid;
 Pid = fork();
 if (pid == 0) {
  Printf("child Process executing \n");
  sleep(5)
 Printf("child Process existing \n");
 }
 else
 {
  wait(NULL);
 Printf("Parent Process resumed after
   child termination \n");
 }
 return 0;
}
```

3) get pid() system call

```
#include <stdio.h>
#include <unistd.h>
int main()
{
 Printf("Process ID: %d \n", getpid
 Printf("Parent Process ID: %d \n
      getppid());
 return 0;
}
```

output :- m/c dir 2 ~~ls~~
contents of directory
one.txt, first.txt, evenmish ....

6) opendir() and readdir() system calls.

```
#include <stdio.h>
#include <dirent.h>
int main (int argc, char* argv[])
{DIR *dir;
struct dirent* entry;
if (argc<2){
  Print ("usage: %s <directory path
  > /n", argv[0]);
  return 1;
}.
dir = opendir (argv[1]);
if (dir= NULL){
   Perror ("opendir").
   return 1;
}.
Printf ("contents of directory : \n");
while ((entry= readdir (dir))!=NULL)
Printf ("%s\n" entry->d_name);
}
close dir (dir);
return 0;
}.
```

Result :- Implementation system calls using has been done & output is verified.

output: Parent Process:

Parent PID = 3555
child PID = 3556

child Process:

child PID: 3556
Parent PID: 3555

Exc: 06
Date: 13-03-26 Implement Process Managment.

fork() system call

```
#include <stdio.h>
#include <unistd.h>
int main()
{pid_t Pid;
Pid = fork();
if (pid < 0){
        printf("Fork failed \n");
}
else if (pid == 0){
Printf ("child Process \n");
Printf ("child PID: %d \n", getpid());
Printf ("Parent PID: %d \n", getppid());
}
else {
 Printf ("Parent Process \n");
Printf ("Parent PID: %d \n", getpid());
Printf ("child PID: %d \n", Pid);
}
return 0;
}
```

b) execl() system call

```
#include <stdio.h>
#include <unistd.h>
int main().
```

output: Parent Process
child Process before exec
ls -l
5.C. 2d.sh 3e.sh
4b.sh

```
{pid_t rid;
pid = fork();
if (pid == 0){
  Printf("Child Process before exec()\n")
  execl("/bin/ls", "ls", NULL);
Printf("this line will not be exec
  \n");
}
else{
  Printf("parent Process \n");
}
return 0;
}
```

Result: Implementation of Process managment has done & output is verified

output (FCFS) :-

Enter number of Process : 2

Enter burst times : 1 3

Process	B.T	WT	TAT
		0	1
P1	1	1	4
P2	3		

Average waiting time :- 0.50

Average Turnaround time : 2.50

Exp 07 — Implementation various CPU schedulingalgorithm using C.

Date :- 20-09-24

FIFO / FCFS (first come first serve)

```
#include <stdio.h>
int main()
{int n,i;
int bt[10],wt[10],tat[10];
float avg_wt=0, avg_tat=0;
Printf ("Enter number of Processes:");
scanf ("%d", &n);
Printf ("Enter burst times: \n");
for (i=0; i<n; i++){
  scanf ("%d", &bt[i]);
}
wt[0]=0;
for(i=1; i<n; i++){
  wt[i] = wt[i-1] + bt[i-1];
}
for (i=0; i<n; i++){
  tat[i]= wt[i]+ bt[i];
  avg_wt += wt[i];
  avg_tat += tat[i];
}
Printf ("\nProcess \tBT \tWT \tTAT\n");
  for (i=0; i<n; i++){
```

output enter number of processes : 2
enter burst times : 10
enter time quantum : 6

Process	BT	WT	TAT
P_1	6	0	6
P_2	5	1	6

Average waiting time : 0.50
Average Turnaround time : 3.50

```
Printf("%d\t %d\t %d\t %d\t\n", i+1, bt[i]
    wt[i], tat[i]);
}
Printf("Average waiting time = %f\t\n", avg_wt);
Printf("Average Turnaround time = %f\t\n",
                                avg_tat);
return 0;
}
```

b) Round Robin Scheduling

```
#include <stdio.h>
int main()
{int n, i, tq;
int bt[10], rt[10], wt[10], tat[10];
int time=0, done;
float avg_wt=0, avg_tat=0;
Printf ("Enter number of Process:");
scanf ("%d", &n);
Printf ("Enter burst time:\n");
  for (i=0; i<n; i++){
    scanf ("%d", &bt[i]);
    rt[i] = bt[i];
    wt[i] =0;
}
Printf ("Enter time quantum:");
```

```
scanf("%d", &tq);
do {
  done = 1;
  for (i=0; i<n; i++){
  if (rt[i] > 0) {
    done = 0;
  if (rt[i] > tq) {
  time += tq;
  rt[i] -= tq;
  }
  else {
    time += rt[i]
    wt[i] = time - bt[i]
      rt[i] = 0;
    }
  }
}while (!done);
for (i=0; i<n; i++){
tat[i] = bt[i] + wt[i];
avg-wt += wt[i];
avg-tat += tat[i];
}
Printf("\n Process | BT | WT | TAT \n");
for (i=0; i<n; i++){
  Printf(" %d | %d | %d | %d \n
```

that is order number of Process
order burst times : 4 3

Process	B.T	W.T	T.A.T
P_1	1	0	1
P_2	3	1	4

Average waiting time : 0.50
Average Turnaround time : 2.5

```
printf("Average waiting time: %f\n",
        avg_wt/n);
printf("Average Turnaround time:
        %f\n", avg_tat/n);
return 0;
}
```

a) SJF (Shortest Job First)

```
#include<stdio.h>
int main()
{int n, i, j;
int bt[10], wt[10], tat[10], temp;
float avg_wt = 0, avg_tat = 0;
Printf("Enter number of Process:")
scanf("%d", &n);
Print f("Enter burst times:\n");
for (i=0; i<n; i++){
  scanf("%d", &bt[i]);
}
for (i=0; i<n-1; i++){
    for (j=i+1; j<n; j++){
      if (bt[i] > bt[j]){
        temp = bt[i];
```

```
        bt[i] = bt[j];
    }
    bt[i] = temp;
  }
}
wt[0] = 0;
for (i=1; i<n; i++) {
    wt[i] = wt[i-1] + bt[i-1];
}
for (i=0; i<n; i++) {
tat[i] = bt[i] + wt[i];
avg_wt += wt[i];
avg_tat += tat[i];
}
Printf("\n Process \t BT \t WT \t TAT \n");
for (i=0; i<n; i++) {
Printf("p%d \t %d \t %d \t %d \n", i+1,
        bt[i], wt[i], tat[i]);
}
Printf("Average waiting Time= %f\n",
        avg_wt/n);
printf("Average Turnaround time= %f\n",
        avg_tat/n);
return 0;
}
```

Result:- Implementing of various cpu scheduling algorithm using c has been done.

Exp:- 08
Date:- 03-04-26

To avoid Deadlock using Banker Algorithm.

```
#include<stdio.h>
int main()
{int n,m,i,j,k;
 int alloc [10] [10], max [10] [10],
need [10]. [10];
int avail [10], finish [10], safe seq [10]
Print f ("Enter number of Process:\n);
scan f ("%d", &n);
Print f ("Enter number of resources:\n);
Scan f ("%d", &m);
Print f ("Enter Allocation matrix:\n);
for (i=0 ; i<n; i++);
   for (i=0 ; i<m; i++);
      scanf ("%d", &alloc [i] [j]);
Print f ("Enter maximum matrix:\n);
for (i=0; i<n; i++)
 for (j=0; i<n; j++)
   scan f ("%d", &max [i] [j])
Print f "Enter Available Resources: \n);
    for (i=0; i<n; i++
    scan f ("%d", &avail [i])
    for (i=0 ; i<n; i++).
```

output

enter number of process : 3

enter no. of resources : 3

enter Allocation matrix

```
1 1 0
2 2 4
4 3 5
2 9 0
6 8 0
7 0 0
```

enter maximum matrix

```
1 1 0
2 2 4
3 3 5
4 6 6
7 8 9
3 9 4
```

enter available Resource =?

```
2 1 4
3 4 3
```

system is in safe state

safe sequence : P0 P1 P2 P4 P3

```
for (i=0; i<m; i++)
  need [i][j] = max [i][j] - alloc [i][j]
for (i=0; i<m; i++)
  finish [i] = 0
int count = 0;
while (count<m) {
  int found = 0;
for (i=0; i<m; i++) {
  if (finish [i] == 0) {
for (j=0; j<n; j++)
  if (need [i][j] > avail [j])
    break;
if (j==n) {
  for (k=0; k<n; k++)
    avail [k] = avail [k] + alloc [i] [k]
    safe seq [count++] = i;
    finish [i] = 1;
    found = 1;
  }
}
}
if (!found) {
  Printf ("\n system is in unsafe
          (Dead Lock Possible)\n");
  return 0;
}
}
```

```
Printf ("\n system is in SAFE state\n
        safe sequence :");
  for (i=0 ; i<n ; i++)
    Printf ("%d "; safe seq [i]);
  Printf ("\n");
  return 0;
}
```

Result : Banker's Algorithm using c is done & output verified.

enter number of blocks : 5

enter size of each block

100, 200, 300, 500, 600

enter no. of Process : 4

enter size of each Process

212 417 112 426

Process	size	Block
P1	212	3
P2	417	3
P3	112	3
P4	426	4

Ex: 19

Date: 3-4-20 Implementing the memory Allocation methods.

a) first fit Program.

```
#include <stdio.h>
int main()
{ int np, nb, i;
  int b[10], p[10], alloc[10];
Print f ("Enter number of blocks:");
scan f ("%d", &nb);
Print f ("Enter size of each block:\n");
  for (i=0; i<nb; i++)
  scan f ("%d", &b[i]);
Print f ("Enter number of Process:");
  scan f ("%d", &np);
Print f ("Enter size of each Process:\n");
  for (i=0; i<np; i++)
    scan f ("%d", &p[i]);
  for (i=0; i<np; i++)
    alloc [i]=-1;
  for (i=0; i<np; i++){
  for (j=0; j<nb; j++){
    if (b[j] >= p[i]){
      alloc[i] = j;
      b[j] -= p[i];
      break;
```

output: enter number of blocks : 5

enter size of each block

100 500 200 300 600

Enter no. of Process : -4

enter size of each Processor :-

312 417 112 426

Process	size	Block
P1	312	5
P2	417	5
P3	112	4
P4	426	3

```
Printf ("\n Process \t size \t Block \n");
  for (i=0; i<np; i++) {
    if (alloc[i] != -1)
Printf ("P %d \t %d \t %d \n", i+1, p[i],
                alloc[i]+1);
else:
  Printf ("P %d \t %d \t Not Allocated \n",
  i+1, p[i]);
  }
  return 0;
  }
```

b) worst fit Program:

```
#include <stdio.h>
int main()
{ int nb, np, i, j;
int b[10], p[10], alloc[10];
int max; pos;
Printf ("Enter number of blocks:");
scanf ("%d", &nb);
Printf ("Enter size of each block:
        \n");
for (i=0; i<nb; i++);
scanf ("%d", &b[i]);
Printf ("Enter number of Processes:")
scanf ("%d", &np).
```

```
Printf ("Enter size of each Process:\n");
for (i=0; i<n; i++)
    scanf ("%d", &p[i]);
for (i=0; i<np; i++)
    alloc [i] = -1;
for (i=0; i<np; i++) {
  max = -1
  pos = -1
  for (j=0; j<nb; j++){
    if (b[j] >= p[i] && b[j] > max){
      max = b [j]
      pos = j;
    }
  }
  if (pos != -1) {
      alloc [i] = pos;
      b [pos] -= p[i];
  }
}
Printf ("\n Process \t size \t Block \n");
for (i=0; i<np; i++) {
    if (alloc [i] != -1)
        Printf ("P%d \t %d \t %d \n", i+1, p[i],
    alloc [i] +1);
    else
      Printf ("P%d \t %d \t Not Allocation \n",
    i+1, p[i]);
}
return 0;
}
```

output: enter number of blocks : 5
Enter size of each block:
100 500 200 300 600
Enter no. of Processes : 4
Enter size of each Process
212 417 112 426

Process	size	Block
P1	212	2
P2	417	5
P3	112	2
P4	426	not allocated

2) Best fit Algorithm:

```
#include<stdio.h>
int main()
{
  int nb, np, i;
  int b[20], p[20], alloc[20];
  int min, Pos;
  Printf ("Enter number of blocks: ");
  scanf ("%d", &nb);
  Printf ("Enter size of each block: \n");
  for (i=0; i<nb; i++)
      scanf ("%d", &b[i]);
  Printf ("Enter number of Processes: ");
  scanf ("%d", &np);
  Printf ("Enter size of each Process: \n");
  for (i=0; i<np; i++)
      scanf ("%d", &p[i]);
  for (i=0; i<np; i++)
      alloc [i] = -1;
  for (i=0; i<np; i++) {
      min = 9999;
      Pos = -1;
      for (j=0; j<nb; j++){
          if (b[j] >= p[i] && b[j] < min)
          {
              min = b[j]
              Pos = j;
          }
          if (pos != -1)
          {
              alloc [i] = pos;
```

```
    b[pos]-=p[i];
  }
}
Printf("\nProcess\t Size\t Block\n");
 for (i=0; i<n p; i++) {
    if (alloc[i] != -1)
        Printf ("p%d \t%d \t%d\n",
i+1, P[i], alloc[i]+1);
  else
    Printf ("p%d \tF%d \tNot Allocated
\n", i+1, P[i]);
 }
return 0;
}
```

Result:- Implementing the memory Allocation methods has done & output is verified.

Completed

17/4/26